

LAPORAN PRAKTIKUM STRUKTUR DATA

Tree dan Graph



disusun Oleh:

Tiara Amalia Insani

NIM 2511532019

Dosen Pengampu : DR. Wahyudi, S.T, M.T

Asisten Praktikum : Jovantri Immanuel Gulo

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum Struktur Data yang berjudul Tree dan Graph. Laporan ini disusun untuk memenuhi salah satu komponen penilaian dalam mata kuliah Struktur Data. Fokus utama dari praktikum ini adalah memahami bagaimana data non-linear diorganisasikan dalam bentuk Tree (Pohon) dan Graph (Graf), serta bagaimana algoritma Breadth First Search (BFS) dan Depth First Search (DFS) diimplementasikan untuk memecahkan masalah penelusuran struktur tersebut. Penyusunan visualisasi berbasis antarmuka grafis (GUI) dalam laporan ini merupakan bagian dari tugas integrasi untuk mempermudah pemahaman perilaku algoritma secara nyata. Penulis menyampaikan terima kasih kepada dosen pengampu dan asisten laboratorium yang telah memberikan bimbingan selama pelaksanaan praktikum. Penulis menyadari laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan untuk perbaikan ke depan.

Padang, 11 Juni 2026

Penyusun

DAFTAR ISI

Contents

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I.....	1
PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat	1
BAB II	2
PEMBAHASAN	2
2.1 Pengertian Tree dan Graph.....	2
2.2 Praktikum	2
2.2.1 Alat dan Bahan	2
2.2.2 Program Class Node.....	2
2.2.3 Program Class BTree	7
2.2.4 Program Class BTree Driver	11
2.2.5 Program Class GraphTraversal.....	15
BAB III.....	22
KESIMPULAN DAN SARAN	22
3.1 Kesimpulan	22
3.2 Saran	22
DAFTAR PUSTAKA.....	23
Penjelasan Tugas	24

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam ilmu komputer, representasi data tidak selalu berbentuk linear seperti *Array*, *List*, *Stack*, atau *Queue*. Untuk menggambarkan hubungan data yang lebih kompleks di dunia nyata—seperti hierarki organisasi, struktur silsilah, jaringan komunikasi, hingga peta rute jalan—diperlukan struktur data non-linear. Dua jenis struktur data non-linear yang paling fundamental adalah *Tree* (Pohon) dan *Graph* (Graf). Setelah data berhasil diorganisasikan ke dalam bentuk *Tree* atau *Graph*, tantangan berikutnya adalah bagaimana mengunjungi atau mencari informasi di dalam struktur tersebut secara efisien. Proses ini dikenal sebagai *traversal* atau penelusuran. Dua algoritma dasar yang paling penting dalam penelusuran ini adalah *Breadth First Search* (BFS) dan *Depth First Search* (DFS).

1.2 Tujuan Praktikum

1. Memahami konsep, karakteristik, dan perbedaan antara struktur data *Tree* dan *Graph*.
2. Menganalisis cara kerja algoritma BFS dalam menelusuri struktur data menggunakan bantuan *Queue*.
3. Menganalisis cara kerja algoritma DFS dalam menelusuri struktur data menggunakan bantuan *Stack*.
4. Membandingkan performa BFS dan DFS dalam hal optimalitas jalur dan total simpul yang dikunjungi.

1.3 Manfaat

Praktikum ini memiliki manfaat untuk membantu mahasiswa memahami *Tree* dan *Graph*, serta pemahaman mendasar mengenai mekanisme penelusuran menggunakan strategi *Breadth First Search* (BFS) dan *Depth First Search* (DFS). Sehingga dapat melatih ketajaman berpikir dan mengasah keterampilan pemecahan masalah (*problem solving*) mahasiswa secara nyata.

BAB II

PEMBAHASAN

2.1 Pengertian Tree dan Graph

Tree adalah struktur data non-linear yang merepresentasikan hubungan hierarki atau bertingkat antara elemen-elemennya. Karakteristik utama yang membedakan *Tree* dari struktur non-linear lainnya adalah tidak diperbolehkannya ada siklus (*cycle*) atau jalur memutar di dalamnya.

Tree selalu dimulai dari satu simpul utama di paling atas yang disebut Root (Akar). Simpul-simpul di bawahnya terhubung secara searah membentuk hubungan antara Parent (Induk) dan Child (Anak). Simpul yang berada di ujung paling bawah dan tidak memiliki anak lagi disebut sebagai Leaf (Daun).

Sedangkan *Graph* adalah struktur data non-linear yang merepresentasikan jaringan jala (*network*) bebas, di mana hubungan antar-elemennya setara dan tidak mengenal hierarki atas-bawah. Berbeda dengan *Tree*, pada *Graph* diperbolehkan adanya siklus (*cycle*), artinya kita bisa berjalan melingkar dan kembali ke simpul asal melalui rute yang berbeda.

2.2 Praktikum

2.2.1 Alat dan Bahan

1. Laptop/computer dengan eclipse terinstal.
2. Materi praktikum mengenai Tree dan Graph

2.2.2 Program Class Node

```
package pekan9_2511532019;

public class Node_2511532019 {
    int data_2019;
    Node_2511532019 left_2019;
    Node_2511532019 right_2019;
    public Node_2511532019 (int data_2019) {
        this.data_2019=data_2019;
        left_2019=null;
        right_2019=null;
    }
    public void setLeft_2019 (Node_2511532019 node_2019) {
        if (left_2019==null)
            left_2019=node_2019;
    }
    public void setRight_2019 (Node_2511532019 node_2019) {
```

```

        if (right_2019==null)
            right_2019=node_2019;
    }
    public Node_2511532019 getLeft_2019() {
        return left_2019;
    }
    public Node_2511532019 getRight_2019() {
        return right_2019;
    }
    public int getData_2019() {
        return data_2019;
    }
    public void setData_2019 (int data_2019) {
        this.data_2019=data_2019;
    }
    void printPreorder_2019 (Node_2511532019 node_2019) {
        if (node_2019==null)
            return;
        System.out.print(node_2019.data_2019+" ");
        printPreorder_2019(node_2019.left_2019);
        printPreorder_2019(node_2019.right_2019);
    }
    void printPostorder_2019 (Node_2511532019 node_2019) {
        if (node_2019==null)
            return;
        printPostorder_2019 (node_2019.left_2019);
        printPostorder_2019 (node_2019.right_2019);
        System.out.print(node_2019.data_2019+" ");
    }
    void printInorder_2019 (Node_2511532019 node_2019) {
        if (node_2019==null)
            return;
        printInorder_2019(node_2019.left_2019);
        System.out.print(node_2019.data_2019+" ");
        printInorder_2019(node_2019.right_2019);
    }
    public String print_2019 () {
        return this.print_2019("",true, "");
    }
    public String print_2019 (String prefix_2019, boolean
isTail_2019, String sb_2019) {
        if (right_2019!=null) {
            right_2019.print_2019 (prefix_2019+(isTail_2019?"|
":" "), false, sb_2019);
        }
        System.out.println(prefix_2019 + (isTail_2019? "\\-- ":"/--
")+data_2019);
        if (left_2019!=null) {
            left_2019.print_2019 (prefix_2019+(isTail_2019?"
":"| "), true, sb_2019);
        }
        return sb_2019;
    }
}

```

- package pekan9_2511532019;
- Menentukan bahwa kelas ini berada dalam package pekan9_2511532019.

- `public class Node_2511532019 {`
→ Mendeklarasikan kelas publik bernama `Node_2511532019`.
- `int data_2019;`
→ Mendeklarasikan variabel bertipe `int` untuk menyimpan data node.
- `Node_2511532019 left_2019;`
→ Mendeklarasikan referensi ke node (anak) kiri.
- `Node_2511532019 right_2019;`
→ Mendeklarasikan referensi ke node (anak) kanan.
- `public Node_2511532019 (int data_2019) {`
→ Membuat konstruktor yang akan dijalankan saat objek node dibuat.
- `this.data_2019=data_2019;`
→ Mengisi atribut `data_2019` milik objek dengan nilai parameter yang diterima.
- `left_2019=null;`
→ Menginisialisasi node kiri dengan nilai `null`.
- `right_2019=null;}`
→ Menginisialisasi node kanan dengan nilai `null`.
- `public void setLeft_2019 (Node_2511532019 node_2019) {`
→ Membuat method untuk menambahkan node kiri.
- `if (left_2019==null)`
→ Mengecek apakah node kiri masih kosong.
- `left_2019=node_2019;}`
→ Mengisi node kiri dengan node yang diberikan.
- `public void setRight_2019 (Node_2511532019 node_2019) {`
→ Membuat method untuk menambahkan node kanan.
- `if (right_2019==null)`
→ Mengecek apakah node kanan masih kosong.
- `right_2019=node_2019;}`
→ Mengisi node kanan dengan node yang diberikan.
- `public Node_2511532019 getLeft_2019() {`
→ Membuat method untuk mengambil node kiri.
- `return left_2019;}`

- Mengembalikan nilai node kiri.
- `public Node_2511532019 getRight_2019() {`
 →Membuat method untuk mengambil node kanan.
- `return right_2019;}`
 →Mengembalikan nilai node kanan.
- `public int getData_2019() {`
 →Membuat method untuk mengambil data node.
- `return data_2019;}`
 →Mengembalikan nilai data node.
- `public void setData_2019 (int data_2019) {`
 →Membuat method untuk mengubah data node.
- `this.data_2019=data_2019;}`
 →Mengganti nilai data node dengan nilai baru.
- `void printPreorder_2019 (Node_2511532019 node_2019) {`
 →Membuat method traversal preorder.
- `if (node_2019==null)`
 →Mengecek apakah node kosong.
- `return;`
 →Menghentikan proses jika node kosong.
- `System.out.print(node_2019.data_2019+" ");`
 →Mencetak data node saat ini.
- `printPreorder_2019(node_2019.left_2019);`
 →Menelusuri subtree kiri secara preorder.
- `printPreorder_2019(node_2019.right_2019);`
 →Menelusuri subtree kanan secara preorder.
- `void printPostorder_2019 (Node_2511532019 node_2019) {`
 →Membuat method traversal postorder.
- `if (node_2019==null)`
 →Mengecek apakah node kosong.
- `return;`
 →Menghentikan proses jika node kosong.
- `printPostorder_2019 (node_2019.left_2019);`

- Menelusuri subtree kiri terlebih dahulu.
- `printPostorder_2019 (node_2019.right_2019);`
→Menelusuri subtree kanan.
- `System.out.print(node_2019.data_2019+" ");`
→Mencetak data node setelah node kiri dan kanan diproses.
- `void printInorder_2019 (Node_2511532019 node_2019) {`
→Membuat method traversal inorder.
- `if (node_2019==null)`
→Mengecek apakah node kosong.
- `return;`
→Menghentikan proses jika node kosong.
- `printInorder_2019(node_2019.left_2019);`
→Menelusuri subtree kiri terlebih dahulu.
- `System.out.print(node_2019.data_2019+" ");`
→Mencetak data node saat ini.
- `printInorder_2019(node_2019.right_2019);`
→Menelusuri subtree kanan.
- `public String print_2019 () {`
→Membuat method untuk mencetak struktur tree.
- `return this.print_2019("",true, "");}`
→Memanggil method `print_2019` dengan parameter awal.
- `public String print_2019 (String prefix_2019, boolean isTail_2019, String sb_2019) {`
→Membuat method rekursif untuk menampilkan struktur tree.
- `if (right_2019!=null) {`
→Mengecek apakah node kanan ada.
- `right_2019.print_2019(prefix_2019+(isTail_2019?"| " : " "), false, sb_2019);}`
→Mencetak subtree kanan terlebih dahulu dengan tambahan indentasi.
- `System.out.println(prefix_2019 + (isTail_2019? "\\-- " : "/--")+data_2019);`
→Mencetak data node beserta simbol cabang tree.
- `if (left_2019!=null) {`

→Mengecek apakah node kiri ada.

- left_2019.print_2019(prefix_2019+(isTail_2019?" ":"| "), true, sb_2019);}

→Mencetak subtree kiri secara rekursif.

- return sb_2019;}}

→Mengembalikan nilai string sb_2019.

2.2.3 Program Class BTree

```
package pekan9_2511532019;

public class BTree_2511532019 {
    private Node_2511532019 root_2019;
    private Node_2511532019 currentNode_2019;
    public BTree_2511532019() {
        root_2019=null;
    }
    public boolean search_2019 (int data_2019) {
        return search_2019 (root_2019, data_2019);
    }
    private boolean search_2019 (Node_2511532019 node_2019, int
data_2019) {
        if (node_2019.getData_2019()==data_2019)
            return true;
        if (node_2019.getLeft_2019()!=null)
            if (search_2019 (node_2019.getLeft_2019(),
data_2019))
                return true;
        if (node_2019.getRight_2019()!= null)
            if(search_2019
(node_2019.getRight_2019(),data_2019))
                return true;
        return false;
    }
    public void printInorder_2019() {
        root_2019.printInorder_2019(root_2019);
    }
    public void printPreorder_2019() {
        root_2019.printPreorder_2019(root_2019);
    }
    public void printPostorder_2019() {
        root_2019.printPostorder_2019(root_2019);
    }
    public Node_2511532019 getRoot_2019() {
        return root_2019;
    }
    public boolean isEmpty_2019() {
        return root_2019==null;
    }
    public int countNodes_2019() {
        return countNodes_2019(root_2019);
    }
    private int countNodes_2019(Node_2511532019 node_2019) {
        int count_2019=1;
    }
}
```

```

        if (node_2019==null) {
            return 0;
        }else {
            count_2019 += countNodes_2019
(node_2019.getLeft_2019());
            count_2019 += countNodes_2019
(node_2019.getRight_2019());
            return count_2019;
        }
    }
    public void print_2019 () {
        root_2019.print_2019();
    }
    public Node_2511532019 getCurrent_2019() {
        return currentNode_2019;
    }
    public void setCurrent_2019 (Node_2511532019 node_2019) {
        this.currentNode_2019=node_2019;
    }
    public void setRoot_2019 (Node_2511532019 root_2019) {
        this.root_2019=root_2019;
    }
}

```

- package pekan9_2511532019;
→Menentukan bahwa kelas ini berada pada package pekan9_2511532019.
- public class BTree_2511532019 {
→Mendeklarasikan kelas publik bernama BTree_2511532019.
- private Node_2511532019 root_2019;
→Mendeklarasikan variabel root_2019 yang digunakan untuk menyimpan node akar (root) dari binary tree.
- private Node_2511532019 currentNode_2019;
→Mendeklarasikan variabel currentNode_2019 untuk menyimpan node yang sedang diproses.
- public BTree_2511532019() {
→Membuat konstruktor kelas BTree_2511532019.
- root_2019=null;}
→Menginisialisasi root dengan nilai null, artinya tree masih kosong.
- public boolean search_2019 (int data_2019) {
→Membuat method untuk mencari data pada binary tree.
- return search_2019 (root_2019, data_2019);}
→Memanggil method pencarian rekursif dimulai dari root.

- `private boolean search_2019 (Node_2511532019 node_2019, int data_2019) {`
→ Membuat method rekursif untuk mencari data pada tree.
- `if (node_2019.getData_2019()==data_2019)`
→ Mengecek apakah data pada node sama dengan data yang dicari.
- `return true;`
→ Mengembalikan nilai true jika data ditemukan.
- `if (node_2019.getLeft_2019()!=null)`
→ Mengecek apakah node kiri ada.
- `if (search_2019 (node_2019.getLeft_2019(), data_2019))`
→ Mencari data secara rekursif pada node kiri.
- `return true;`
→ Mengembalikan nilai true jika data ditemukan pada node kiri.
- `if (node_2019.getRight_2019()!= null)`
→ Mengecek apakah node kanan ada.
- `if(search_2019 (node_2019.getRight_2019(),data_2019))`
→ Mencari data secara rekursif pada node kanan.
- `return true;`
→ Mengembalikan nilai true jika data ditemukan pada node kanan.
- `return false;}`
→ Mengembalikan nilai false jika data tidak ditemukan.
- `public void printInorder_2019() {`
→ Membuat method untuk menampilkan tree dengan traversal inorder.
- `root_2019.printInorder_2019(root_2019);}`
→ Memanggil method inorder mulai dari root.
- `public void printPreorder_2019() {`
→ Membuat method untuk menampilkan tree dengan traversal preorder.
- `root_2019.printPreorder_2019(root_2019);}`
→ Memanggil method preorder mulai dari root.
- `public void printPostorder_2019() {`
→ Membuat method untuk menampilkan tree dengan traversal postorder.
- `root_2019.printPostorder_2019(root_2019);}`

- Memanggil method postorder mulai dari root.
- `public Node_2511532019 getRoot_2019() {`
 - Membuat method untuk mengambil root tree.
- `return root_2019;}`
 - Mengembalikan nilai root.
- `public boolean isEmpty_2019() {`
 - Membuat method untuk memeriksa apakah tree kosong.
- `return root_2019==null;}`
 - Mengembalikan nilai true jika root bernilai null.
- `public int countNodes_2019() {`
 - Membuat method untuk menghitung jumlah node.
- `return countNodes_2019(root_2019);}`
 - Memanggil method perhitungan node secara rekursif mulai dari root.
- `private int countNodes_2019(Node_2511532019 node_2019) {`
 - Membuat method rekursif untuk menghitung jumlah node.
- `int count_2019=1;`
 - Menginisialisasi penghitung node dengan nilai 1 untuk node saat ini.
- `if (node_2019==null) {`
 - Mengecek apakah node kosong.
- `return 0;`
 - Mengembalikan nilai 0 jika node kosong.
- `}else {`
 - Menjalankan blok jika node tidak kosong.
- `count_2019 += countNodes_2019 (node_2019.getLeft_2019());`
 - Menambahkan jumlah node pada bagian kiri tree.
- `count_2019 += countNodes_2019 (node_2019.getRight_2019());`
 - Menambahkan jumlah node pada bagian kanan tree.
- `return count_2019;}}`
 - Mengembalikan total jumlah node.
- `public void print_2019 () {`
 - Membuat method untuk menampilkan struktur tree.
- `root_2019.print_2019();}`

- Memanggil method cetak tree dari root.
- `public Node_2511532019 getCurrent_2019() {`
 →Membuat method untuk mengambil node yang sedang aktif.
- `return currentNode_2019;}`
 →Mengembalikan nilai `currentNode_2019`.
- `public void setCurrent_2019 (Node_2511532019 node_2019) {`
 →Membuat method untuk mengubah node yang sedang aktif.
- `this.currentNode_2019=node_2019;}`
 →Mengisi `currentNode_2019` dengan node yang diberikan.
- `public void setRoot_2019 (Node_2511532019 root_2019) {`
 →Membuat method untuk mengubah root tree.
- `this.root_2019=root_2019;}}`
 →Mengisi root dengan node yang diberikan.

2.2.4 Program Class BTree Driver

```
package pekan9_2511532019;

public class BTreeDriver_2511532019 {
    public static void main (String[] args) {
        //membuat Pohon
        BTree_2511532019 tree_2019=new BTree_2511532019();
        System.out.print("Jumlah simpul awal pohon: ");
        System.out.println(tree_2019.countNodes_2019());
        //menambahkan simpul data
        Node_2511532019 root_2019=new Node_2511532019(1);
        //menjadikan simpul 1 sebagai root
        tree_2019.setRoot_2019(root_2019);
        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree_2019.countNodes_2019());
        Node_2511532019 node2_2019 = new Node_2511532019 (2);
        Node_2511532019 node3_2019 = new Node_2511532019 (3);
        Node_2511532019 node4_2019 = new Node_2511532019 (4);
        Node_2511532019 node5_2019 = new Node_2511532019 (5);
        Node_2511532019 node6_2019 = new Node_2511532019 (6);
        Node_2511532019 node7_2019 = new Node_2511532019 (7);
        Node_2511532019 node8_2019 = new Node_2511532019 (8);
        Node_2511532019 node9_2019 = new Node_2511532019 (9);
        root_2019.setLeft_2019(node2_2019);
        node2_2019.setLeft_2019(node4_2019);
        node2_2019.setRight_2019(node5_2019);
        node4_2019.setRight_2019(node8_2019);
        root_2019.setRight_2019(node3_2019);
        node3_2019.setLeft_2019(node6_2019);
        node3_2019.setRight_2019(node7_2019);
        node6_2019.setLeft_2019(node9_2019);
        //set root
    }
}
```

```

        tree_2019.setCurrent_2019(tree_2019.getRoot_2019());
        System.out.println("Menampilkan simpul terakhir: ");

        System.out.println(tree_2019.getCurrent_2019().getData_2019());
        System.out.println("Jumlah simpul; setelah simpul 7
ditambahkan");
        System.out.println(tree_2019.countNodes_2019());
        System.out.println("InOrder: ");
        tree_2019.printInorder_2019();
        System.out.println("\nPreorder: ");
        tree_2019.printPreorder_2019();
        System.out.println("\nPostorder: ");
        tree_2019.printPostorder_2019();
        System.out.println("\nMenampilkan simpul dalam bentuk
pohon");
        tree_2019.print_2019();
    }
}

```

- package pekan9_2511532019;
→ Menentukan bahwa kelas ini berada pada package pekan9_2511532019.
- public class BTreeDriver_2511532019 {
→ Mendeklarasikan kelas publik bernama BTreeDriver_2511532019.
- public static void main (String[] args) {
→ Method utama yang pertama kali dijalankan saat program dieksekusi.
- BTree_2511532019 tree_2019=new BTree_2511532019();
→ Membuat objek Binary Tree bernama tree_2019.
- System.out.print("Jumlah simpul awal pohon: ");
→ Menampilkan teks "Jumlah simpul awal pohon:".
- System.out.println(tree_2019.countNodes_2019());
→ Menampilkan jumlah simpul pada tree yang masih kosong.
- Node_2511532019 root_2019=new Node_2511532019(1);
→ Membuat simpul baru dengan data bernilai 1.
- tree_2019.setRoot_2019(root_2019);
→ Menjadikan node bernilai 1 sebagai root tree.
- System.out.println("Jumlah simpul jika hanya ada root");
→ Menampilkan teks keterangan jumlah simpul saat hanya ada root.
- System.out.println(tree_2019.countNodes_2019());
→ Menampilkan jumlah simpul yang ada pada tree.
- Node_2511532019 node2_2019 = new Node_2511532019 (2);
→ Membuat simpul baru dengan data 2.

- Node_2511532019 node3_2019 = new Node_2511532019 (3);
→Membuat simpul baru dengan data 3.
- Node_2511532019 node4_2019 = new Node_2511532019 (4);
→Membuat simpul baru dengan data 4.
- Node_2511532019 node5_2019 = new Node_2511532019 (5);
→Membuat simpul baru dengan data 5.
- Node_2511532019 node6_2019 = new Node_2511532019 (6);
→Membuat simpul baru dengan data 6.
- Node_2511532019 node7_2019 = new Node_2511532019 (7);
→Membuat simpul baru dengan data 7.
- Node_2511532019 node8_2019 = new Node_2511532019 (8);
→Membuat simpul baru dengan data 8.
- Node_2511532019 node9_2019 = new Node_2511532019 (9);
→Membuat simpul baru dengan data 9.
- root_2019.setLeft_2019(node2_2019);
→Menjadikan node 2 sebagai node kiri dari root.
- node2_2019.setLeft_2019(node4_2019);
→Menjadikan node 4 sebagai node kiri dari node 2.
- node2_2019.setRight_2019(node5_2019);
→Menjadikan node 5 sebagai node kanan dari node 2.
- node4_2019.setRight_2019(node8_2019);
→Menjadikan node 8 sebagai node kanan dari node 4.
- root_2019.setRight_2019(node3_2019);
→Menjadikan node 3 sebagai node kanan dari root.
- node3_2019.setLeft_2019(node6_2019);
→Menjadikan node 6 sebagai node kiri dari node 3.
- node3_2019.setRight_2019(node7_2019);
→Menjadikan node 7 sebagai node kanan dari node 3.
- node6_2019.setLeft_2019(node9_2019);
→Menjadikan node 9 sebagai node kiri dari node 6.
- tree_2019.setCurrent_2019(tree_2019.getRoot_2019());
→Mengatur node yang sedang aktif menjadi root tree.

- `System.out.println("Menampilkan simpul terakhir: ");`
→ Menampilkan teks keterangan.
- `System.out.println(tree_2019.getCurrent_2019().getData_2019());`
→ Menampilkan data dari node yang sedang aktif.
- `System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");`
→ Menampilkan keterangan jumlah simpul setelah penambahan node.
- `System.out.println(tree_2019.countNodes_2019());`
→ Menampilkan jumlah seluruh simpul pada tree.
- `System.out.println("InOrder: ");`
→ Menampilkan tulisan "InOrder:".
- `tree_2019.printInorder_2019();`
→ Menampilkan traversal Inorder.
- `System.out.println("\nPreorder: ");`
→ Menampilkan tulisan "Preorder:" pada baris baru.
- `tree_2019.printPreorder_2019();`
→ Menampilkan traversal Preorder.
- `System.out.println("\nPostorder: ");`
→ Menampilkan tulisan "Postorder:" pada baris baru.
- `tree_2019.printPostorder_2019();`
→ Menampilkan traversal Postorder.
- `System.out.println("\nDmenampilkan simpul dalam bentuk pohon");`
→ Menampilkan keterangan bahwa simpul akan ditampilkan dalam bentuk pohon.
- `tree_2019.print_2019();}}`
→ Menampilkan struktur tree dalam bentuk diagram pohon.

Output

```

Jumlah simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
Menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|      /--7
|      /--3
|      | \-- 6
|      | \-- 9
|-- 1
|   /--5
|-- 2
|   /--8
|-- 4

```

2.2.5 Program Class GraphTraversal

```

package pekan9_2511532019;

import java.util.Queue;
import java.util.Set;
import java.util.Map;
import java.util.List;
import java.util.HashMap;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedList;

public class GraphTraversal_2511532019 {
    private Map<String, List<String>> graph = new HashMap<>();
    //Menambahkan edge (graf tak berarah)
    public void addEdge_2019 (String node1_2019, String node2_2019) {
        graph.putIfAbsent(node1_2019, new ArrayList<>());
        graph.putIfAbsent(node2_2019, new ArrayList<>());
        graph.get (node1_2019).add(node2_2019);
        graph.get(node2_2019).add(node1_2019);
    }
    //menampilkan graf awal
    public void printGraph_2019() {
        System.out.println("Graf Awal (Adjacency List): ");
        for(String node_2019 : graph.keySet()) {
            System.out.print(node_2019+" -> ");
            List<String> neighbors_2019=graph.get(node_2019);
            System.out.println(String.join(", ",
neighbors_2019));
        }
        System.out.println();
    }
}

```

```

    }
    //DFS rekursif
    public void dfs_2019 (String start_2019) {
        Set<String> visited_2019=new HashSet<>();
        System.out.println("Penelusuran DFS: ");
        dfsHelper_2019(start_2019,visited_2019);
        System.out.println();
    }
    private void dfsHelper_2019(String current_2019,
Set<String>visited_2019) {
        if (visited_2019.contains(current_2019)) return;
        visited_2019.add(current_2019);
        System.out.print(current_2019+ " ");
        for (String
neighbor_2019:graph.getDefault(current_2019,new ArrayList<>())) {
            dfsHelper_2019 (neighbor_2019, visited_2019);
        }
    }
    //BFS iteratif
    public void bfs_2019 (String start_2019) {
        Set <String>visited_2019=new HashSet<>();
        Queue <String> queue_2019= new LinkedList<>();
        queue_2019.add(start_2019);
        visited_2019.add(start_2019);
        System.out.println("Penelusuran BFS: ");
        while (!queue_2019.isEmpty()) {
            String current_2019=queue_2019.poll();
            System.out.print(current_2019+" ");
            for (String
neighbor_2019:graph.getDefault(current_2019, new ArrayList<>())) {
                if (!visited_2019.contains(neighbor_2019)){
                    queue_2019.add(neighbor_2019);
                    visited_2019.add(neighbor_2019);
                }
            }
        }
        System.out.println();
    }
    //Main
    public static void main (String [] args) {
        GraphTraversal_2511532019 graph_2019=new
GraphTraversal_2511532019();
        //contoh graf : A-B, A-C, B-D, B-E
        graph_2019.addEdge_2019("A","B");
        graph_2019.addEdge_2019("A","C");
        graph_2019.addEdge_2019("B","D");
        graph_2019.addEdge_2019("B","E");
        //cetak graf awal
        System.out.println("Graf Awal adalah: ");
        graph_2019.printGraph_2019();
        //Lakukan penelusuran
        graph_2019.dfs_2019("A");
        graph_2019.bfs_2019("A");
    }
}

```

- `package pekan9_2511532019;`
→ Menentukan bahwa kelas ini berada dalam package `pekan9_2511532019`.
- `import java.util.Queue;`
→ Mengimpor kelas `Queue` untuk menyimpan data dalam bentuk antrian.
- `import java.util.Set;`
→ Mengimpor kelas `Set` untuk menyimpan kumpulan data tanpa duplikasi.
- `import java.util.Map;`
→ Mengimpor kelas `Map` untuk menyimpan pasangan key dan value.
- `import java.util.List;`
→ Mengimpor kelas `List` untuk menyimpan kumpulan data secara berurutan.
- `import java.util.HashMap;`
→ Mengimpor kelas `HashMap` sebagai implementasi dari `Map`.
- `import java.util.ArrayList;`
→ Mengimpor kelas `ArrayList` sebagai implementasi dari `List`.
- `import java.util.HashSet;`
→ Mengimpor kelas `HashSet` sebagai implementasi dari `Set`.
- `import java.util.LinkedList;`
→ Mengimpor kelas `LinkedList` yang digunakan sebagai `Queue`.
- `public class GraphTraversal_2511532019 {`
→ Mendeklarasikan kelas publik bernama `GraphTraversal_2511532019`.
- `private Map<String, List<String>> graph = new HashMap<>();`
→ Membuat variabel `graph` untuk menyimpan graf dalam bentuk Adjacency List.
- `public void addEdge_2019 (String node1_2019, String node2_2019) {`
→ Membuat method untuk menambahkan hubungan antara dua simpul.
- `graph.putIfAbsent(node1_2019, new ArrayList<>());`
→ Menambahkan simpul pertama ke graf jika belum ada.
- `graph.putIfAbsent(node2_2019, new ArrayList<>());`
→ Menambahkan simpul kedua ke graf jika belum ada.
- `graph.get(node1_2019).add(node2_2019);`
→ Menambahkan node kedua ke daftar tetangga node pertama.
- `graph.get(node2_2019).add(node1_2019);}`

- Menambahkan node pertama ke daftar tetangga node kedua.
- `public void printGraph_2019() {`
 - Membuat method untuk menampilkan isi graf.
- `System.out.println("Graf Awal (Adjacency List): ");`
 - Menampilkan judul graf.
- `for(String node_2019 : graph.keySet()) {`
 - Melakukan perulangan untuk setiap simpul dalam graf.
- `System.out.print(node_2019+" -> ");`
 - Menampilkan nama simpul.
- `List<String> neighbors_2019=graph.get(node_2019);`
 - Mengambil daftar tetangga dari simpul tersebut.
- `System.out.println(String.join(" ", neighbors_2019));}`
 - Menampilkan semua tetangga yang dimiliki simpul.
- `System.out.println();}`
 - Mencetak baris kosong.
- `public void dfs_2019 (String start_2019) {`
 - Membuat method DFS dengan simpul awal tertentu.
- `Set<String> visited_2019=new HashSet<>();`
 - Membuat himpunan untuk menyimpan simpul yang sudah dikunjungi.
- `System.out.println("Penelusuran DFS: ");`
 - Menampilkan judul DFS.
- `dfsHelper_2019(start_2019,visited_2019);`
 - Memanggil method bantuan DFS.
- `System.out.println();}`
 - Mencetak baris baru setelah DFS selesai.
- `private void dfsHelper_2019(String current_2019, Set<String>visited_2019) {`
 - Membuat method rekursif untuk DFS.
- `if (visited_2019.contains(current_2019)) return;`
 - Jika simpul sudah dikunjungi maka proses dihentikan.
- `visited_2019.add(current_2019);`
 - Menandai simpul sebagai sudah dikunjungi.

- `System.out.print(current_2019+ " ");`
→Menampilkan simpul yang sedang dikunjungi.
- `for (String neighbor_2019:graph.getDefault(current_2019,new ArrayList<>())) {`
→Melakukan perulangan untuk semua tetangga simpul saat ini.
- `dfsHelper_2019 (neighbor_2019, visited_2019);}}`
→Mengunjungi tetangga secara rekursif.
- `public void bfs_2019 (String start_2019) {`
→Membuat method BFS dengan simpul awal tertentu.
- `Set <String>visited_2019=new HashSet<>();`
→Membuat himpunan untuk menyimpan simpul yang telah dikunjungi.
- `Queue <String> queue_2019= new LinkedList<>();`
→Membuat antrean untuk proses BFS.
- `queue_2019.add(start_2019);`
→Menambahkan simpul awal ke antrean.
- `visited_2019.add(start_2019);`
→Menandai simpul awal sebagai sudah dikunjungi.
- `System.out.println("Penelusuran BFS: ");`
→Menampilkan judul BFS.
- `while (!queue_2019.isEmpty()) {`
→Perulangan akan berjalan selama antrean tidak kosong.
- `String current_2019=queue_2019.poll();`
→Mengambil dan menghapus elemen paling depan dari antrean.
- `System.out.print(current_2019+" ");`
→Menampilkan simpul yang sedang diproses.
- `for (String neighbor_2019:graph.getDefault(current_2019, new ArrayList<>())) {`
→Melakukan perulangan pada semua tetangga simpul saat ini.
- `if (!visited_2019.contains(neighbor_2019)){`
→Mengecek apakah tetangga belum pernah dikunjungi.
- `queue_2019.add(neighbor_2019);`
→Menambahkan tetangga ke antrean.

- `visited_2019.add(neighbor_2019);}}`
→Menandai tetangga sebagai sudah dikunjungi.
- `System.out.println();`
→Mencetak baris baru setelah BFS selesai.
- `public static void main (String [] args) {`
→Method utama yang pertama kali dijalankan saat program dieksekusi.
- `GraphTraversal_2511532019` `graph_2019=new`
`GraphTraversal_2511532019();`
→Membuat objek graf baru.
- `graph_2019.addEdge_2019("A","B");`
→Menambahkan sisi antara simpul A dan B.
- `graph_2019.addEdge_2019("A","C");`
→Menambahkan sisi antara simpul A dan C.
- `graph_2019.addEdge_2019("B","D");`
→Menambahkan sisi antara simpul B dan D.
- `graph_2019.addEdge_2019("B","E");`
→Menambahkan sisi antara simpul B dan E.
- `System.out.println("Graf Awal adalah: ");`
→Menampilkan judul graf awal.
- `graph_2019.printGraph_2019();`
→Menampilkan graf dalam bentuk Adjacency List.
- `graph_2019.dfs_2019("A");`
→Melakukan penelusuran DFS dimulai dari simpul A.
- `graph_2019.bfs_2019("A");}}`
→Melakukan penelusuran BFS dimulai dari simpul A.

Output

```
Graf Awal adalah:  
Graf Awal (Adjacency List):  
A -> B, C  
B -> A, D, E  
C -> A  
D -> B  
E -> B  
  
Penelusuran DFS:  
A B D E C  
Penelusuran BFS:  
A B C D E
```


BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data *Tree* dan *Graph* memiliki perbedaan karakteristik yang sangat mendasar pada bentuk hubungannya, di mana *Tree* merepresentasikan hubungan hierarki bertingkat yang bebas dari jalur memutar atau siklus, sedangkan *Graph* merepresentasikan jaringan jala bebas yang memperbolehkan adanya siklus antar simpul. Karakteristik penelusuran dari algoritma *Breadth First Search* (BFS) dan *Depth First Search* (DFS) ditentukan secara mutlak oleh jenis struktur data penampung sementara, di mana BFS dikendalikan oleh sifat antrean FIFO (*First In First Out*) pada Queue sedangkan DFS dikendalikan oleh sifat tumpukan LIFO (*Last In First Out*) pada Stack.

3.2 Saran

1. lebih banyak berlatih dalam mengimplementasikan Tree dan Graph agar semakin memahami konsepnya
2. Perlu meningkatkan ketelitian dalam penulisan kode untuk menghindari kesalahan (*error*) saat program dijalankan.

DAFTAR PUSTAKA

- [1] M. T. Goodrich, R. Tamassia, dan M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ: Wiley, 2014.

Penjelasan Tugas

Program Class PetaSekolah

```
package pekan9_2511532019;

import javax.swing.*.*;
import javax.swing.text.DefaultHighlighter;
import javax.swing.text.Highlighter;
import java.awt.*.*;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.LinkedList;
import java.util.Stack;
import java.util.Set;

public class PetaSekolah_2511532019 extends JFrame {

    private JComboBox<String> awal_2019;
    private JComboBox<String> tujuan_2019;
    private JButton bfs_2019;
    private JButton dfs_2019;
    private JButton reset_2019;
    private JTextArea graphArea_2019;
    private JTextArea jalur_2019;
    private JTextArea nodeDikunjungi_2019;
    private JLabel jumlahNode_2019;
    private Map<String, List<String>> graphData_2019 = new HashMap<>();
    private String petaTeks_2019;

    private void addEdge_2019(String node1_2019, String node2_2019) {
        graphData_2019.putIfAbsent(node1_2019, new ArrayList<>());
        graphData_2019.putIfAbsent(node2_2019, new ArrayList<>());

        graphData_2019.get(node1_2019).add(node2_2019);
        graphData_2019.get(node2_2019).add(node1_2019);
    }

    private void buatGraph_2019() {
        addEdge_2019("Lapangan_Voli", "L._Basket");
        addEdge_2019("Lapangan_Voli", "Tata_Busana");

        addEdge_2019("L._Basket", "Lap._Takraw");
        addEdge_2019("L._Basket", "Ruang_Guru");
        addEdge_2019("Lap._Takraw", "Uks");
        addEdge_2019("Tata_Busana", "Ptsp");
        addEdge_2019("Tata_Busana", "Kantin");
        addEdge_2019("Ptsp", "Ruang_Guru");
        addEdge_2019("Ruang_Guru", "Koperasi");
        addEdge_2019("Ruang_Guru", "Aula");
        addEdge_2019("Ruang_Guru", "Kantin");
    }
}
```

```

        addEdge_2019("Koperasi", "Aula");
        addEdge_2019("Koperasi", "Mesjid");
        addEdge_2019("Aula", "Uks");
        addEdge_2019("Aula", "Kantin");
        addEdge_2019("Kantin", "Mesjid");
        addEdge_2019("Uks", "Hall");
        addEdge_2019("Uks", "Mesjid");
        addEdge_2019("Hall", "Mesjid");
    }

    private void displayGraph_2019(List<String> urutan_2019) {
        Highlighter h_2019 = graphArea_2019.getHighlighter();
        h_2019.removeAllHighlights();

        Highlighter.HighlightPainter painter_2019 = new
        DefaultHighlighter.DefaultHighlightPainter(new Color(255, 150, 150));
        String teksPeta_2019 = graphArea_2019.getText();

        for (String node_2019 : urutan_2019) {
            int index_2019 = teksPeta_2019.indexOf(node_2019);
            while (index_2019 >= 0) {
                try {
                    h_2019.addHighlight(index_2019, index_2019 +
node_2019.length(), painter_2019);
                } catch (Exception ex) {
                    ex.printStackTrace();
                }
                index_2019 = teksPeta_2019.indexOf(node_2019, index_2019
+ 1);
            }
        }
    }

    private void displayPath_2019(ArrayList<String> path_2019,
ArrayList<String> urutan_2019, String tipe_2019) {
        jalur_2019.setText("Jalur " + tipe_2019 + " : " +
(path_2019.isEmpty() ? "Tidak ditemukan" : String.join(" -> ",
path_2019)));
        nodeDikunjungi_2019.setText("Node Dikunjungi : " + String.join("
-> ", urutan_2019));
        jumlahNode_2019.setText("Jumlah Node Dikunjungi / Dieksplorasi :
" + urutan_2019.size());
    }

    private void resetGraph_2019() {
        graphArea_2019.getHighlighter().removeAllHighlights();
        jalur_2019.setText("Jalur : ");
        nodeDikunjungi_2019.setText("Node Dikunjungi : ");
        jumlahNode_2019.setText("Jumlah Node Dikunjungi / Dieksplorasi :
0");
    }

    // Method Wajib: BFS
    private void bfs_2019() {
        ArrayList<String> urutan_2019 = new ArrayList<>();
        String start_2019 = awal_2019.getSelectedItem().toString();
    }

```

```

        String tujuan_2019Str =
tujuan_2019.getSelectedItemAt().toString();

        Queue<String> queue_2019 = new LinkedList<>();
        Set<String> visited_2019 = new HashSet<>();
        Map<String, String> parent_2019 = new HashMap<>();

        queue_2019.add(start_2019);
        visited_2019.add(start_2019);
        parent_2019.put(start_2019, null);

        boolean found = false;

        while (!queue_2019.isEmpty()) {
            String current_2019 = queue_2019.poll();
            urutan_2019.add(current_2019);

            if (current_2019.equals(tujuan_2019Str)) {
                found = true;
                break;
            }

            List<String> neighbors =
graphData_2019.getDefault(current_2019, new ArrayList<>());
            for (String neighbor_2019 : neighbors) {
                if (!visited_2019.contains(neighbor_2019)) {
                    visited_2019.add(neighbor_2019);
                    parent_2019.put(neighbor_2019, current_2019);
                    queue_2019.add(neighbor_2019);
                }
            }
        }

        ArrayList<String> path_2019 = new ArrayList<>();
        if (found) {
            String current_2019 = tujuan_2019Str;
            while (current_2019 != null) {
                path_2019.add(0, current_2019);
                current_2019 = parent_2019.get(current_2019);
            }
        }

        displayPath_2019(path_2019, urutan_2019, "BFS");
        displayGraph_2019(urutan_2019);
    }

    // Method Wajib: DFS
    private void dfs_2019() {
        ArrayList<String> urutan_2019 = new ArrayList<>();
        String start_2019 = awal_2019.getSelectedItemAt().toString();
        String tujuan_2019Str =
tujuan_2019.getSelectedItemAt().toString();

        Stack<String> stack_2019 = new Stack<>();
        Set<String> visited_2019 = new HashSet<>();
        Map<String, String> parent_2019 = new HashMap<>();

        stack_2019.push(start_2019);
    }

```

```

        parent_2019.put(start_2019, null);

        boolean found = false;

        while (!stack_2019.isEmpty()) {
            String current_2019 = stack_2019.pop();

            if (!visited_2019.contains(current_2019)) {
                visited_2019.add(current_2019);
                urutan_2019.add(current_2019);

                if (current_2019.equals(tujuan_2019Str)) {
                    found = true;
                    break;
                }

                List<String> neighbors_2019 =
graphData_2019.getDefault(current_2019, new ArrayList<>());
                for (int i = neighbors_2019.size() - 1; i >= 0; i--) {
                    String next_2019 = neighbors_2019.get(i);
                    if (!visited_2019.contains(next_2019)) {
                        parent_2019.put(next_2019, current_2019);
                        stack_2019.push(next_2019);
                    }
                }
            }
        }

        ArrayList<String> path_2019 = new ArrayList<>();
        if (found) {
            String current_2019 = tujuan_2019Str;
            while (current_2019 != null) {
                path_2019.add(0, current_2019);
                current_2019 = parent_2019.get(current_2019);
            }
        }

        displayPath_2019(path_2019, urutan_2019, "DFS");
        displayGraph_2019(urutan_2019);
    }

    public PetaSekolah_2511532019() {
        setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
        setSize(850, 720);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(null);
        buatGraph_2019();

        JLabel lblAwal_2019 = new JLabel("Lokasi Awal :");
        lblAwal_2019.setBounds(20, 20, 100, 25);
        add(lblAwal_2019);

        String[] lokasi_2019 = {
            "Lapangan_Voli", "L._Basket", "Lap._Takraw",
            "Tata_Busana", "Ptsp", "Ruang_Guru", "Koperasi",
            "Aula", "Kantin", "Uks", "Hall", "Mesjid"
        }
    }

```



```

graphArea_2019.setText(petaTeks_2019);
panelGraph_2019.add(new JScrollPane(graphArea_2019));
add(panelGraph_2019);

JLabel hasil_2019 = new JLabel("Hasil Pencarian :");
hasil_2019.setBounds(20, 425, 200, 25);
add(hasil_2019);

jalur_2019 = new JTextArea("Jalur : ");
jalur_2019.setBounds(20, 455, 790, 40);
jalur_2019.setEditable(false);
jalur_2019.setLineWrap(true);
jalur_2019.setWrapStyleWord(true);
jalur_2019.setBackground(getBackground());
add(jalur_2019);

nodeDikunjungi_2019 = new JTextArea("Node dikunjungi : ");
nodeDikunjungi_2019.setBounds(20, 505, 790, 50);
nodeDikunjungi_2019.setEditable(false);
nodeDikunjungi_2019.setLineWrap(true);
nodeDikunjungi_2019.setWrapStyleWord(true);
nodeDikunjungi_2019.setBackground(getBackground());
add(nodeDikunjungi_2019);

jumlahNode_2019 = new JLabel("Jumlah Node dikunjungi /
Dieksplorasi : 0");
jumlahNode_2019.setBounds(20, 565, 400, 25);
add(jumlahNode_2019);

bfs_2019.addActionListener(e -> bfs_2019());
dfs_2019.addActionListener(e -> dfs_2019());
reset_2019.addActionListener(e -> resetGraph_2019());
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new PetaSekolah_2511532019().setVisible(true);
    });
}
}

```

- package pekan9_2511532019;
→ Menentukan folder/package tempat class ini berada.
- import javax.swing.*;
→ Mengambil semua komponen GUI Swing (JFrame, JButton, dll).
- import javax.swing.text.DefaultHighlighter;
import javax.swing.text.Highlighter;
→ Untuk fitur highlight (memberi warna teks).
- import java.awt.*;
→ Untuk layout, warna, font.

- `import java.util.*;`
→ Untuk struktur data (ArrayList, HashMap, Queue, Stack, Set)
- `public class PetaSekolah_2511532019 extends JFrame {`
→ Membuat GUI window (JFrame) bernama GraphGUI.
- `private JComboBox<String> awal_2019;`
`private JComboBox<String> tujuan_2019;`
→ Dropdown untuk pilih (start node, tujuan node)
- `private JButton bfs_2019;`
`private JButton dfs_2019;`
`private JButton reset_2019;`
→ Tombol (BFS, DFS, Reset)
- `private JTextArea graphArea_2019;`
→ Area untuk menampilkan ASCII graph
- `private JTextArea jalur_2019;`
`private JTextArea nodeDikunjungi_2019;`
`private JLabel jumlahNode_2019;`
→ Output hasil (jalur hasil pencarian, urutan node dikunjungi, jumlah node)
- `private Map<String, List<String>>> graphData_2019 = new`
`HashMap<>();`
→ Struktur graph: node → list tetangga
- `private String petaTeks_2019;`
→ Menyimpan ASCII map graph
- `private void addEdge_2019(String node1_2019, String node2_2019)`
→ membuat hubungan antar node
- `graphData_2019.putIfAbsent(node1_2019, new ArrayList<>());`
→ jika node1 belum ada → buat list kosong
- `graphData_2019.putIfAbsent(node2_2019, new ArrayList<>());`
→ jika node2 belum ada → buat list kosong
- `graphData_2019.get(node1_2019).add(node2_2019);`
→ node1 terhubung ke node2
- `graphData_2019.get(node2_2019).add(node1_2019);`

- node2 juga terhubung ke node1 (dua arah)
- `addEdge_2019("Lapangan_Voli", "L._Basket");`
→ Lapangan_Voli terhubung ke L._Basket
- `Highlighter h_2019 = graphArea_2019.getHighlighter();`
→ ambil sistem highlight dari text area
- `h_2019.removeAllHighlights();`
→ hapus warna sebelumnya
- `Highlighter.HighlightPainter painter_2019 =`
`new DefaultHighlighter.DefaultHighlightPainter(new Color(255, 150,`
`150));`
→ membuat warna pink untuk highlight
- `String teksPeta_2019 = graphArea_2019.getText();`
→ ambil isi ASCII graph
- `for (String node_2019 : urutan_2019)`
loop semua node hasil BFS/DFS
- `int index_2019 = teksPeta_2019.indexOf(node_2019);`
→ cari posisi teks node di ASCII
- `h_2019.addHighlight(index_2019, index_2019 + node_2019.length(),`
`painter_2019);`
→ kasih warna ke teks node
- `jalur_2019.setText(...)`
→ tampilkan jalur hasil BFS/DFS
- `String.join(" -> ", path_2019)`
→ gabungkan list jadi:
- `nodeDikunjungi_2019.setText(...)`
→ tampilkan urutan node dikunjungi
- `jumlahNode_2019.setText(...)`
→ tampilkan jumlah node yang dikunjungi
- `graphArea_2019.getHighlighter().removeAllHighlights();`
→ hapus warna highlight
- `jalur_2019.setText("Jalur : ");`
→ reset output jalur

- `nodeDikunjungi_2019.setText("Node Dikunjungi : ");`
→reset node
- `jumlahNode_2019.setText("... : 0");`
→reset jumlah node
- `ArrayList<String> urutan_2019 = new ArrayList<>();`
→simpan urutan kunjungan
- `Queue<String> queue_2019 = new LinkedList<>();`
→antrian BFS
- `Set<String> visited_2019 = new HashSet<>();`
→simpan node yang sudah dikunjungi
- `Map<String, String> parent_2019 = new HashMap<>();`
→simpan jalur balik
- `queue_2019.add(start_2019);`
`visited_2019.add(start_2019);`
`parent_2019.put(start_2019, null);`
→mulai dari node awal
- `while (!queue_2019.isEmpty())`
→selama masih ada node
- `String current_2019 = queue_2019.poll();`
→ambil node paling depan
- `urutan_2019.add(current_2019);`
→catat node yang dikunjungi
- `if (current_2019.equals(tujuan_2019Str))`
→kalau sudah sampai tujuan → stop
- `List<String> neighbors = graphData_2019.getDefault(current_2019,`
`new ArrayList<>());`
→ambil semua tetangga node
- `if (!visited_2019.contains(neighbor_2019))`
→kalau belum pernah dikunjungi
- `visited_2019.add(neighbor_2019);`
`parent_2019.put(neighbor_2019, current_2019);`
`queue_2019.add(neighbor_2019);`

- tandai, simpan parent, masukkan ke queue
- `Stack<String> stack_2019 = new Stack<>();`
→ stack untuk DFS
- `stack_2019.push(start_2019);`
→ mulai dari node awal
- `while (!stack_2019.isEmpty())`
→ selama masih ada node
- `String current_2019 = stack_2019.pop();`
→ ambil node terakhir masuk
- `if (!visited_2019.contains(current_2019))`
→ kalau belum dikunjungi
- `visited_2019.add(current_2019);`
`urutan_2019.add(current_2019);`
→ tandai dan simpan urutan
- `for (int i = neighbors_2019.size() - 1; i >= 0; i--)`
→ loop dari belakang (penting DFS)
- `stack_2019.push(next_2019);`
`parent_2019.put(next_2019, current_2019);`
→ masuk stack + simpan parent
- `setTitle(...)`
→ judul window
- `setSize(...)`
→ ukuran window
- `setLayout(null);`
→ pakai layout manual
- `buatGraph_2019();`
→ bangun graph sebelum GUI tampil
- `JComboBox lokasi_2019`
→ daftar semua node
- `bfs_2019.addActionListener(e -> bfs_2019());`
→ klik BFS jalankan fungsi BFS
- `dfs_2019.addActionListener(e -> dfs_2019());`

→klik DFS jalankan DFS

- `reset_2019.addActionListener(e -> resetGraph_2019());`

→reset GUI

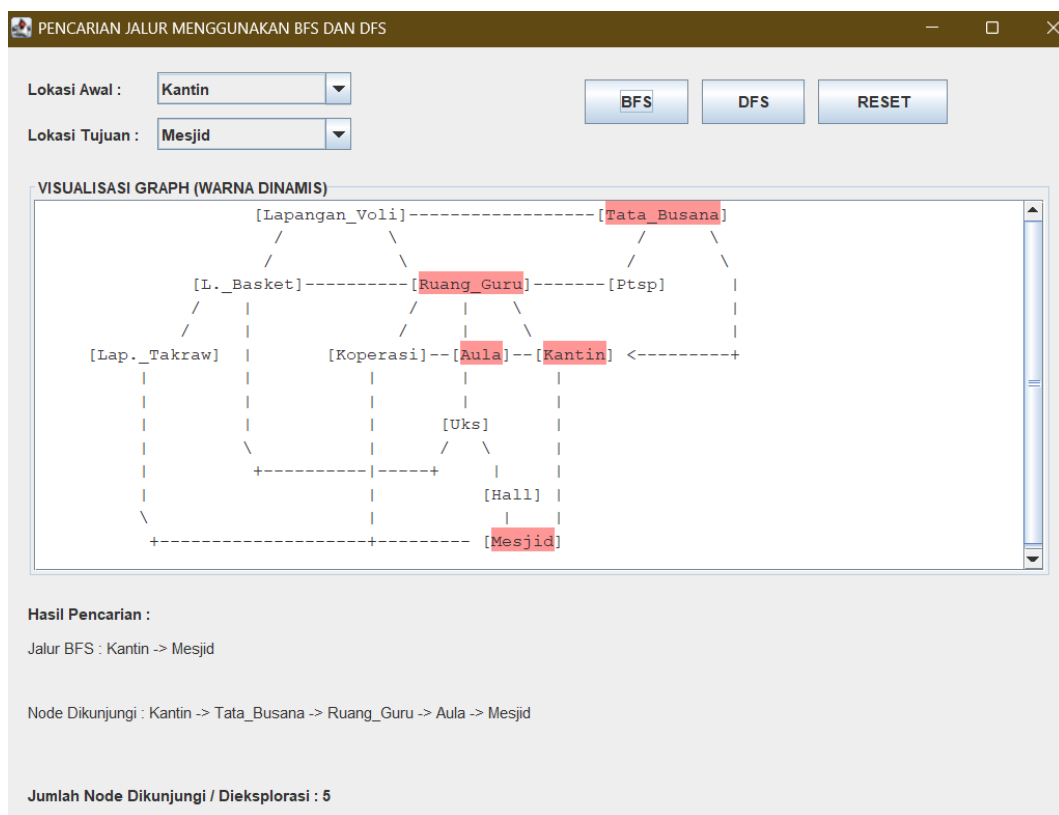
- `graphArea_2019.setText(petaTeks_2019);`

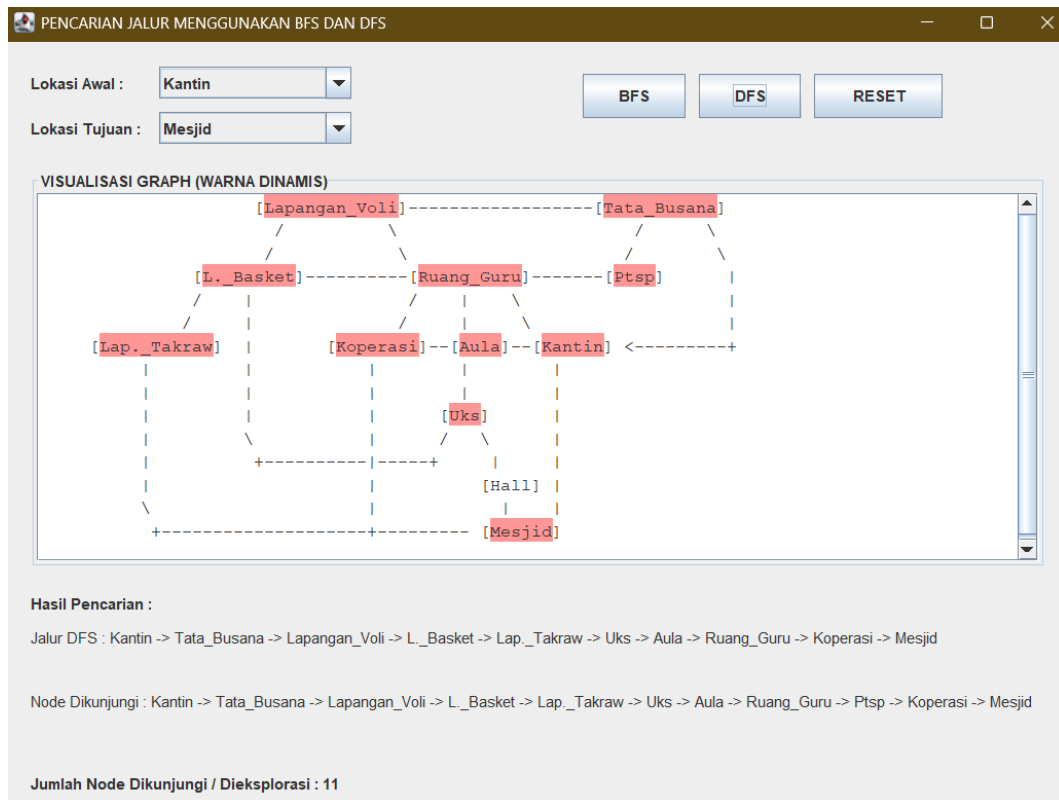
→tampilkan ASCII graph

- `SwingUtilities.invokeLater(...)`

→menjalankan GUI di thread Swing (wajib)

Output





Deskripsi Peta

Implementasi graf pada aplikasi pencarian jalur ini menggunakan model Graf Tidak Berarah tanpa bobot. Graf ini memiliki 12 Node yaitu Lapangan_Voli, L._Basket, Lap._Takraw, Tata_Busana, Ptsp, Ruang_Guru, Koperasi, Aula, Kantin, Uks, Hall, dan Mesjid. Sementara itu, hubungan antar node dijalin oleh 19 Edge (Jalur) yang memungkinkan pergerakan dilakukan secara timbal balik (dua arah). Struktur ketetanggaan ini menciptakan beberapa lintasan alternatif.

Hasil Eksekusi dan Perbandingan Jalur (Studi Kasus: Kantin → Mesjid)

Pada pengujian nyata yang ditunjukkan oleh gambar, pencarian dilakukan dengan menetapkan lokasi awal di Kantin dan lokasi tujuan di Mesjid. Melalui metode Breadth First Search (BFS) yang menggunakan antrean FIFO (*First In First Out*), komputer secara bertahap menyisir seluruh node tetangga lapis demi lapis berdasarkan radius jaraknya. Dari Kantin, BFS mengeksplorasi node-node terdekatnya terlebih dahulu, menghasilkan urutan eksplorasi (*Node Dikunjungi*) sebanyak 5 lokasi, yaitu Kantin, Tata_Busana, Ruang_Guru, Aula, dan Mesjid. Karena keunggulannya yang selalu menjamin jalur terpendek pada graf tanpa

bobot, BFS sukses menemukan rute langsung paling optimal, yaitu Jalur BFS: Kantin -> Mesjid (hanya melewati 1 *edge* langsung).

Sebaliknya, pengujian menggunakan algoritma Depth First Search (DFS) memperlihatkan perilaku pencarian mendalam yang sangat kontras akibat karakteristik tumpukan LIFO (*Last In First Out*). Didorong oleh urutan pembacaan tetangga secara terbalik di dalam kode, DFS justru melewati jalur langsung dan malah tersesat ke cabang terjauh terlebih dahulu. Akibatnya, DFS harus berputar mengitari hampir seluruh isi peta dengan mengeksplorasi 11 lokasi sekaligus, dengan urutan kunjungan: Kantin, Tata_Busana, Lapangan_Voli, L._Basket, Lap._Takraw, Uks, Aula, Ruang_Guru, Ptsp, Koperasi dan Mesjid. Hasil akhir rute yang dibentuk oleh DFS menjadi sangat panjang dan tidak efisien, yaitu Jalur DFS: Kantin -> Tata_Busana -> Lapangan_Voli -> L._Basket -> Lap._Takraw -> Uks -> Aula -> Ruang_Guru -> Koperasi -> Mesjid.

Perbandingan

Dari perbandingan visual kedua gambar tersebut, dapat disimpulkan bahwa untuk kasus pencarian dari Kantin menuju Mesjid, BFS jauh lebih unggul dan efisien dibandingkan DFS. BFS berhasil menemukan rute satu langkah langsung dengan hanya mengeksplorasi 5 node, sementara DFS terperangkap dalam karakteristiknya yang menyusuri satu cabang, memaksa sistem melakukan operasi *backtracking* sehingga mengeksplorasi hingga 11 node dan menghasilkan rute memutar yang tidak optimal. Kasus ini membuktikan secara nyata bahwa BFS sangat konsisten dalam menemukan rute terpendek (*shortest path*), sedangkan DFS hanya fokus menemukan rute pertama yang penting sampai ke tujuan tanpa memedulikan efisiensi panjang jalur.